TI202 - Algorithmique et Structures de données 1

TD4 - Chaînes de caractères (2h)

Consignes:
- Le travail est **individuel**.
- Le travail est à faire sur papier.
- Prenez le temps de lire et de comprendre les questions avant de commencer à répondre.
- Il est fortement recommandé de se référer aux cours et aux supports de cours pour répondre aux questions.

Acquis d'apprentissage:
À la fin de ce TD, vous serez capable de:
- Écrire des programmes simples manipulant des chaîne de caractères.
- Faire la différence entre une chaîne de caractères et un tableau de caractères.
- Utiliser les boucles pour parcourir, puis effectuer des opérations sur les chaînes de caractères.
- Implémenter les premiers algorithmes classiques sur les chaînes de caractères.

Rappel sur les chaînes de caractères:
- Une chaîne de caractères est une suite de caractères.
- En C, une chaîne de caractères est un tableau de caractères terminé par le caractère nul `'\0'`.

Échauffement

Exercice 1. [Première chaîne] Écrire un programme qui affiche la chaîne de caractères `"Hello world"` où cette chaîne est stockée dans une variable de votre programme.

Exemple : *L'exécution de votre programme produira en sortie* `"Hello world"`.

Exercice 2. [Salutations] Alice et Bob ont une manière particulière de converser sur leur application de messagerie préférée. Alice commence toujours une conversation toujours par un `"he..ey"` (C'est la chaîne de caractères `"hey"` avec un ou plusieurs caractères `'e'` à la suite) et Bob répond obligatoirement avec un `"he..ey"` dont le nombre de `'e'` est le double du nombre de `'e'` du `"he..ey"` d'Alice.

Écrire un programme qui prend en entrée une chaîne de caractères `s` de taille $3 \leq n \leq 100$ contenant le `"he..ey"` d'Alice (on suppose que la chaîne commence toujours par un unique `'h'` suivi par un ou plusieurs `'e'` et se termine toujours par un unique `'y'`; puis le programme affichera la réponse de Bob.

Exemple : *Si l'entrée est* `"heeeey"`, *la sortie sera* `"heeeeeeeey"`.

Exercice 3. [Fameux whisky] Un jour, Alice dit à Bob que dans la phrase suivante, toutes les lettres de l'alphabet apparaissent au moins une fois: "Buvez de ce whisky que le patron juge fameux". Bob n'est pas convaincu et demande à Alice de lui prouver.

Compléter le programme suivant afin qu'il affiche la fréquence de chaque lettre de l'alphabet dans la phrase donnée.

```c
#include <stdio.h>

#define ALPH_SIZE 26

int main() {
    char str[] = "Buvez de ce whisky que le patron juge fameux";
    int freq[ALPH_SIZE] = {0};

    /* Votre logique ici */
    // ...

    /* Affichage du résultat */
    for (int i = 0; i < 26; ++i) {
        printf("%c: %d\n", 'a' + i, freq[i]);
    }

    return 0;
}
```

Indication : *Notez que la première lettre de la phrase est une majuscule.*

Renforcement

Dans les exercices suivants, vous pouvez utiliser la fonction `strlen` de la bibliothèque `string.h` qui prend en paramètre une chaîne de caractères et qui retourne sa longueur (le nombre de caractères qui le compose en ne comptant pas le caractère `'\0'`). Par exemple, `strlen("hello")` retourne 5. De plus, chaque *mot* considéré dans les exercices suivants sera une chaîne de caractères ne comportant que des lettres.

Exercice 4. [Palindrome] Alice a récemment découvert une passion pour les mots. Elle sait qu'un palindrome est mot qui se lit de la même manière de gauche à droite et de droite à gauche. Par exemple, "radar", "kayak" sont des palindromes. Étant donné, une petite liste de mots, aidez Alice à compter le nombre de palindromes dans cette liste.

Complétez le programme suivant pour qu'il affiche le nombre de palindromes dans la liste de mots.

```c
#include <stdio.h>
/* Ajoutez ici les bibliothèques nécessaires */
// ...

#define N 10
#define STR_SIZE 100

int main() {

    char words[N][STR_SIZE + 1] = {
        "radar",
        "kayak",
        "hello",
        "world",
        "madam",
        "racecar",
        "civic",
        "level",
        "rotor",
        "noon"
    };
```

```
    /* Votre logique ici */
    // ...

    /* Affichage du résultat */
    printf("Number of palindromes: %d\n", ...);

    return 0;
}
```

Exercice 5. [Anagrammes] Alice et Bob aiment jouer à un jeu où ils doivent trouver si deux mots sont des anagrammes. Deux mots sont des anagrammes si l'un peut être obtenu en réarrangeant les lettres de l'autre. Par exemple, "listen" et "silent" sont des anagrammes.

Aidez-les à écrire un programme qui détermine si deux mots sont des anagrammes en complétez le programme suivant. Le programme doit afficher `"YES"` si les mots sont des anagrammes et `"NO"` sinon. On supposera que les deux mots sont de la même longueur.

```
#include <stdio.h>

#define STR_SIZE 100
#define ALPH_SIZE 26

int main () {
    char a[STR_SIZE + 1];
    char b[STR_SIZE + 1];

    scanf("%s", a);
    scanf("%s", b);

    /* Votre logique ici */
    // ...

    /* Affichage du résultat */
    if (/* Condition */) {
        printf("YES\n");
    } else {
        printf("NO\n");
    }

    return 0;
}
```

Exercice 6. Alice se souvient que lors d'un récent contrôle de programmation, on lui a demandé d'écrire un programme qui vérifie si une chaîne de caractère `sub` était une sous-chaîne d'une chaîne de caractère `s`, et qui afficher la position de la première occurrence dans ce cas-ci. Elle se rappelle également qu'elle avait réussi cet exercice haut la main. Cette fois-ci, elle voudrait écrire un programme qui affiche toutes les positions de la sous-chaîne `sub` dans la chaîne `s`. Le programme affichera `-1` si la sous-chaîne n'est pas présente dans la chaîne.

On suppose que les deux chaînes auront une longueur maximale de 100 caractères, et que `s` est nécessairement plus longue que `sub`. Aidez Alice à compléter le programme suivant.

```
#include <stdio.h>
#include <string.h>

#define STR_SIZE 100

```

```c
int main() {
    char s[STR_SIZE + 1];
    char sub[STR_SIZE + 1];

    scanf("%s", s);
    scanf("%s", sub);

    int n = strlen(s);
    int m = strlen(sub);

    /* Votre logique ici */
    // ...

    return 0;
}
```

Exemple : *Si* `s = "abracadabra"` *et* `sub = "abra"`, *le programme doit afficher* `"0 7"`. *Si* `s = "abracadabra"` *et* `sub = "cad"`, *le programme doit afficher* `"4"`. *Si* `s = "abracadabra"` *et* `sub = "dab"`, *le programme doit afficher* `"-1"`.

Exercice 7. [Pied Pipper] Alice s'amuse à compresser des chaînes de caractères. Elle a remarqué que dans une chaîne de caractères, si un caractère est répété plusieurs fois, elle peut le remplacer par le caractère suivi du nombre de répétitions. Par exemple, la chaîne "aaabbbcc" peut être compressée en "a3b3c2". Le principe de compression est le suivant: chaque nouveau caractère rencontré est ajouté à la chaîne compressée, et le nombre de répétitions est ajouté si le caractère est répété plus d'une fois; sinon, le caractère est simplement ajouté.

Aidez Alice à écrire un programme qui prend une chaîne de caractères en entrée et qui stocke la chaîne compressée dans une autre chaîne de caractères, et qui affiche cette chaîne compressée. Complétez alors le programme suivant.

```c
#include <stdio.h>

#define STR_SIZE 100

int main () {
    char str[STR_SIZE + 1];
    char ans[STR_SIZE + 1];

    scanf("%s", str);

    /* Votre logique ici */
    // ...

    printf("%s\n", ans);
    return 0;
}
```

Exemple : *Si l'entrée est la chaîne* `"aaabbbcc"`, *le programme devra afficher* `"a3b3c2"`. *Si l'entrée est la chaîne* `"hello"`, *le programme devra afficher* `"hel2o"`.

Approfondissement

Exercice 8. [Conversion en nombre] Écrire un programme qui prend en entrée une chaîne de caractères `str` représentant un nombre entier positif, et qui affiche la somme de ce nombre et son inverse. L'inverse d'un

nombre est obtenu en inversant l'ordre des chiffres du nombre. Par exemple, l'inverse de 123 est 321. On suppose que la chaîne de caractères contient au plus 10 chiffres.

Exemple : *Si l'entrée est la chaîne* `"123"`*, le programme devra afficher l'entier* `444` *car* $123 + 321 = 444$.

Exercice 9. [Distance d'édition] La *distance d'édition* entre deux chaînes de caractères est le nombre minimum d'opérations nécessaires pour transformer une chaîne en une autre et inversement. Les opérations possibles sont les suivantes:

- Insérer un caractère.
- Supprimer un caractère.
- Remplacer un caractère.

Le calcul de la distance d'édition entre deux chaînes est un problème classique en informatique qui s'applique par exemple à la correction orthographique, à la comparaison de fichiers, à la bio-informatique, etc.

Prenons par exemple les chaînes "kitten" et "sitting". La distance d'édition entre ces deux chaînes est 3, car on peut transformer "kitten" en "sitting" en remplaçant 'k' par 's', en remplaçant 'e' par 'i', et en insérant un 'g' à la fin.

Pour calculer la distance d'édition entre deux chaînes de caractères `a` et `b`, on peut on peut construire une matrice d'édition D de taille $(n+1) \times (p+1)$ entre les deux chaînes (où n est la longueur de `a` et p est la longueur de `b`).

Dans cette matrice, chaque élément $D[i][j]$ de la matrice, on stockera la distance d'édition pour transformer la sous-chaîne `a[0..i-1]` en la sous-chaîne `b[0..j-1]`, pour tout $0 \leq i \leq n$ et $0 \leq j \leq p$. Dans la matrice D, on a donc autant de lignes que de caractères dans la chaîne `a` plus un et autant de colonnes que de caractères dans la chaîne `b` plus un. La ligne et la colonne supplémentaires correspondent à la sous-chaîne vide.

Pour l'exemple des chaînes "kitten" et "sitting", la matrice D est la suivante:

	''	's'	'i'	't'	't'	'i'	'n'	'g'
''								
'k'								
'i'								
't'								
't'								
'e'								
'n'								

(a) Dans quel élément de la matrice D est stockée la distance d'édition entre les chaînes `a` et `b`?

(b) On remarque qu'en partant de la sous-chaîne vide de `a`, on peut construire la chaîne `b` en insérant les caractères de `b` un par un. De même, on peut retrouver la sous-chaîne vide de `a` en supprimant les caractères de `b` un par un. Exprimez respectivement en fonction de i et j la distance d'édition pour transformer la sous-chaîne vide en la sous-chaîne `a[0..i-1]` et la sous-chaîne vide en la sous-chaîne `b[0..j-1]`.

(c) Déduire les valeurs des $D[i][0]$ et $D[0][j]$ pour tout $0 \geq i \geq n$ et $0 \geq j \geq p$.

(d) Afin de remplir la matrice, il existe trois opérations possibles pour passer de la sous-chaîne `a[0..i-1]` à la sous-chaîne `b[0..j-1]`: insérer un caractère, supprimer un caractère, ou remplacer un caractère. Le calcul de $D[i][j]$ peut donc se faire comme suit:

- Si `a[i-1] == b[j-1]`, alors la distance d'édition pour transformer la sous-chaîne `a[0..i-1]` en la sous-chaîne `b[0..j-1]` est la même que la distance d'édition pour transformer la sous-chaîne `a[0..i-2]` en la sous-chaîne `b[0..j-2]`.

- Sinon, la distance d'édition pour transformer la sous-chaîne `a[0..i-1]` en la sous-chaîne `b[0..j-1]` est 1 plus le minimum entre la distance d'édition pour transformer la sous-chaîne `a[0..i-2]` en la sous-chaîne `b[0..j-1]`, la distance d'édition pour transformer la sous-chaîne `a[0..i-1]` en la sous-chaîne `b[0..j-2]`, et la distance d'édition pour transformer la sous-chaîne `a[0..i-2]` en la sous-chaîne `b[0..j-2]`.

En supposant que les valeurs de $D[i-1][j]$, $D[i][j-1]$ et $D[i-1][j-1]$ sont connues, pour une certaine valeur de i et de j. Exprimez en fonction de ces valeurs la valeur de $D[i][j]$, pour tout $1 \leq i \leq n$ et $1 \leq j \leq p$.

(e) Écrire un programme qui calcule la distance d'édition entre deux chaînes de caractères `a` et `b`, en utilisant la matrice D décrite précédemment. On suppose que les deux chaînes ont une longueur maximale de 100 caractères.

Exemple : *Si les entrées sont* `"kitten"` *et* `"sitting"`*, le programme devra afficher* `3`*. Si les entrées sont* `"hello"` *et* `"world"`*, le programme devra afficher* `4`*. Si les entrées sont* `"geek"` *et* `"greek"`*, le programme devra afficher* `1`*.*